7

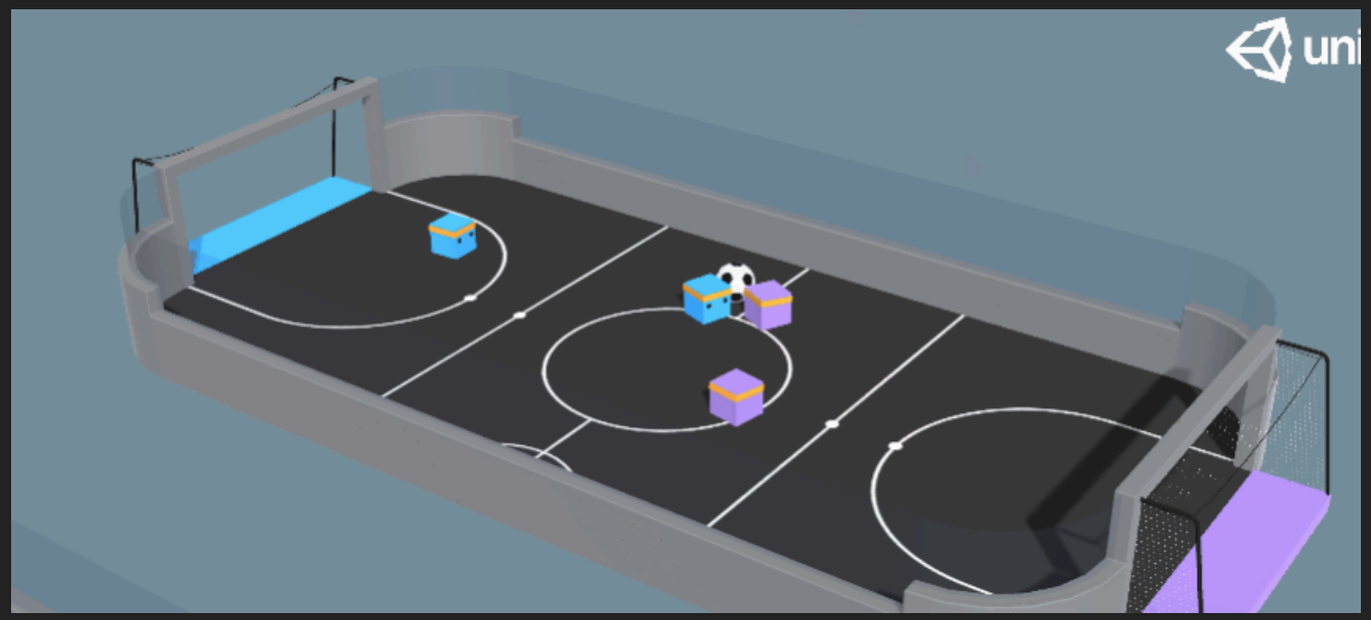
APRENDIZAJE POR REFUERZO MULTIAGENTE (PARTE 1)

INTRODUCCIÓN

- Hasta ahora aprendimos a entrenar agentes en un sistema de agente único donde dicho agente estaba solo en su entorno: no interactuaba ni cooperaba con otros agentes.
- Aun así el sistema de agente único es útil para una gran cantidad de aplicaciones.

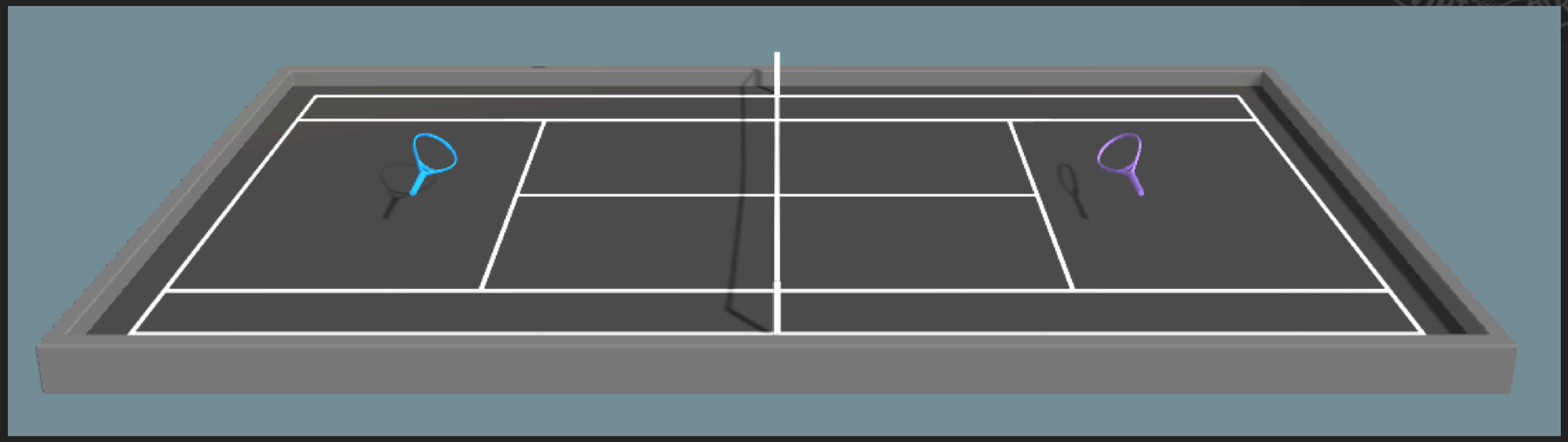
INTRODUCCIÓN

- El objetivo de MARL es entrenar agentes que puedan interactuar con humanos y/o con otros agentes y que puedan adaptarse, colaborar o competir.
- MARL plantea la situación en la que múltiples agentes comparten e interactúan en un entorno común.
- Ejemplos:
- Un almacén donde un enjambre de robots necesitan navegar para cargar y descargar paquetes.
- Una carretera o una intersección con varios vehículos autónomos.
- Casas inteligentes que intentan programar actividades que consumen energía, de tal manera que equilibren la carga de energía a lo largo del día, en lugar de solo en las horas pico.



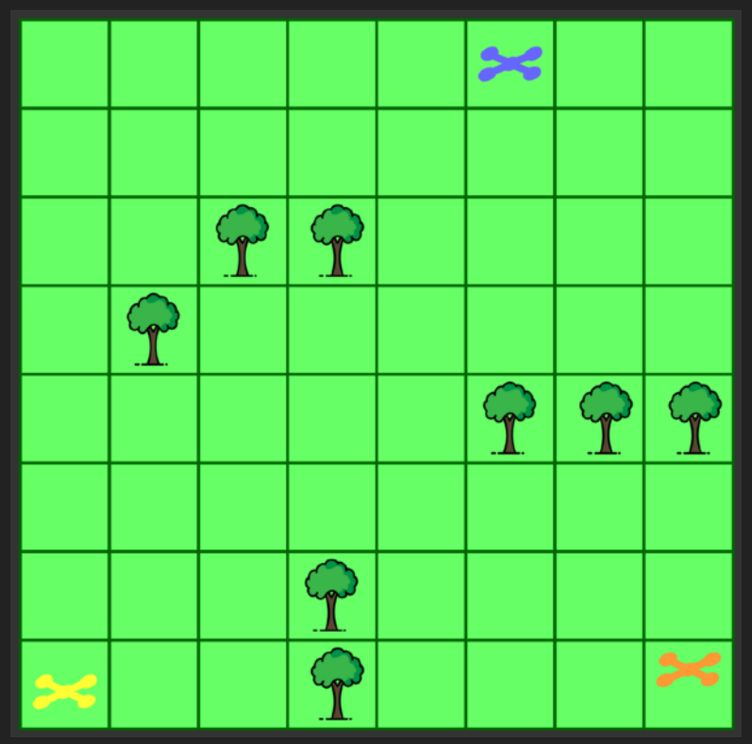
TIPOS DE ENTORNOS MULTIAGENTE

- Entornos cooperativos: donde sus agentes necesitan maximizar los beneficios comunes.
  - ✓ Ejemplo: en un almacén, los robots deben colaborar para cargar y descargar los paquetes de manera eficiente (lo más rápido posible) .
- Entornos competitivos/adversarios: en este caso, el agente desea maximizar sus beneficios minimizando los del oponente.
  - ✓ Ejemplo: en un partido de tenis, cada agente quiere vencer al otro agente .
- Mezcla de adversario y cooperativo: en el fútbol varios agentes necesitan cooperar entre sí y vencer al equipo oponente.



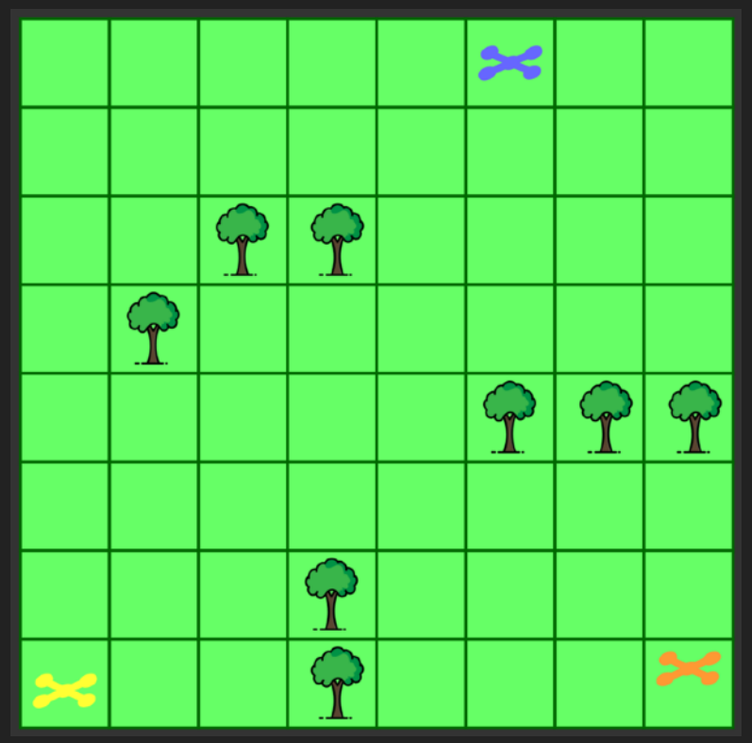
EJEMPLO: TRES DRONES EN UNA FINCA

- Deben fotografiar toda la finca en el menor tiempo posible.
- Deben cubrir cada uno de los cuadrados vacíos al menos una vez.
- Existen cuadrados que representan la ubicación de obstáculos. Los drones no pueden ocupar estos espacios.



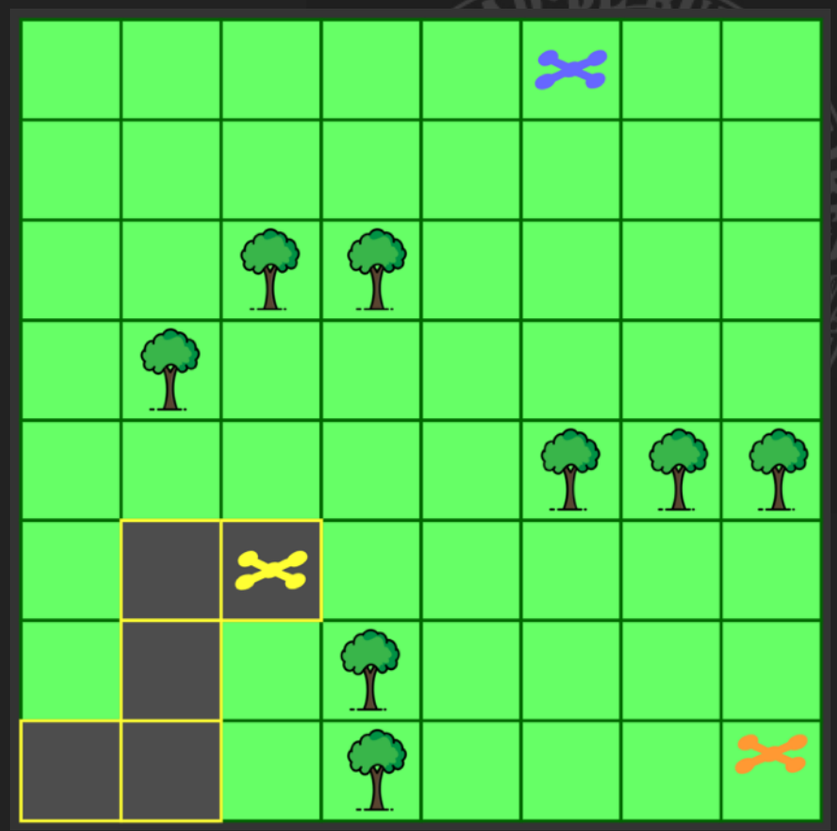
EJEMPLO: TRES DRONES EN UNA FINCA

- En cada paso de tiempo cada dron:
  - ✓ Observa su posición en la finca y dónde están los obstáculos.
  - ✓ Mantiene un mapa de cobertura de dónde ha estado.
  - ✓ Determina una de las 5 acciones a tomar: Norte, Sur, Este, Oeste, o simplemente esperar en su lugar actual hasta el siguiente paso de tiempo.



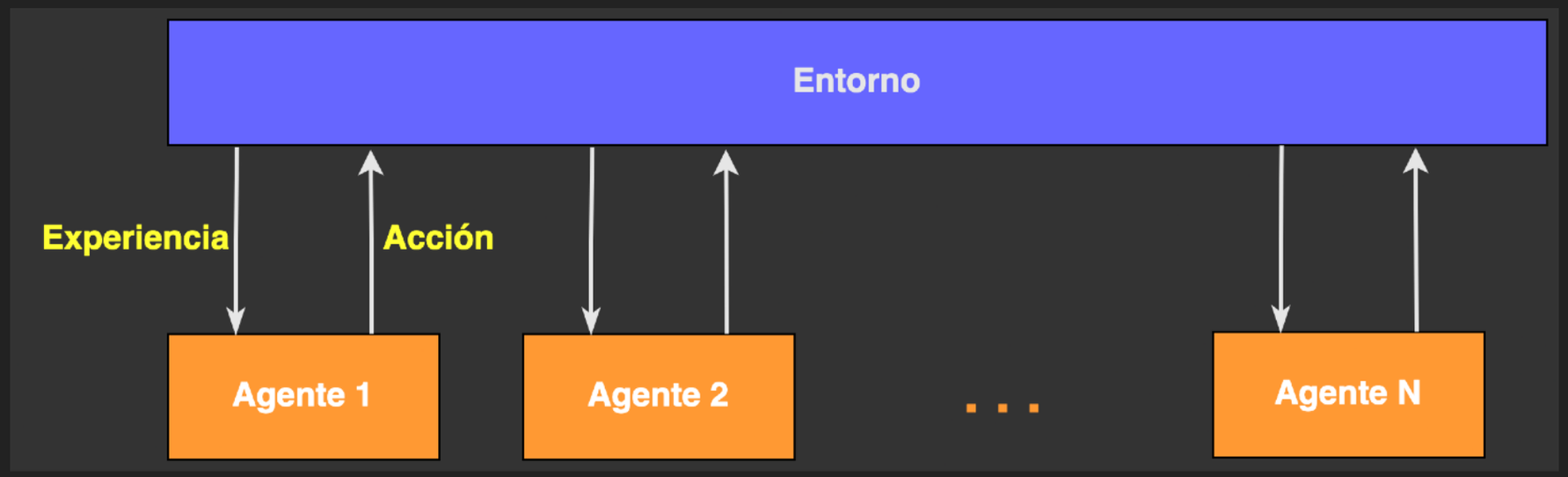
EJEMPLO – RECOMPENSAS

- En cada paso de tiempo, cada dron se debería mover a un cuadrado que aún no ha sido cubierto.
- +1 por moverse a una celda previamente no visitada.
- -0.5 por colisión (obstáculo u otro dron).
- -0.05 –> cuadrado ya cubierto.
- -0.1 –> no moverse.
- +100 por cobertura completa del área. Esta recompensa se comparte entre todos los agentes que están cooperando entre sí una vez que se cubre toda la finca.



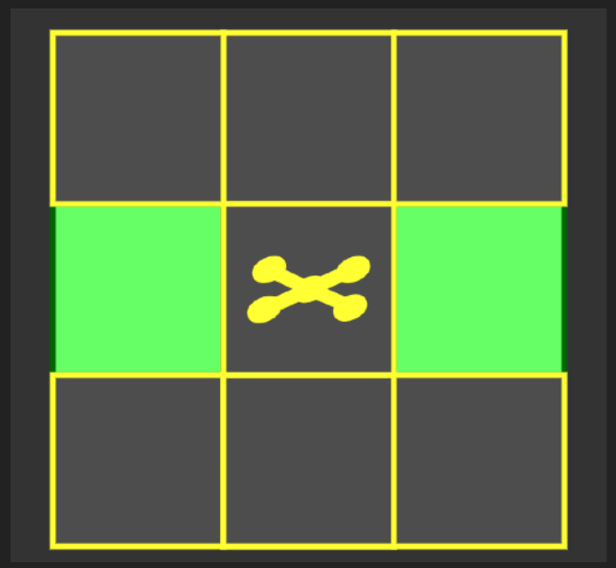
ESCENARIO 1 – ARQUITECTURA DESCENTRALIZADA

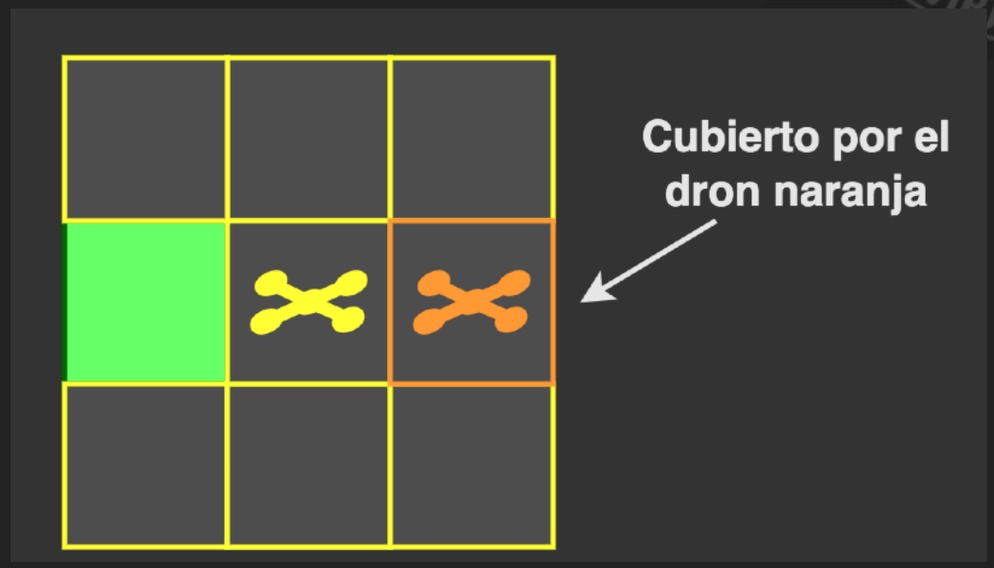
- Cada agente se entrena independientemente de los demás. Cada dron aprende a cubrir la mayor parte posible de la finca sin tener en cuenta lo que están haciendo los otros drones.
- No se comparte información entre los agentes. Están completamente solos para aprender.
- Ventaja:
  - ✓ Como no se comparte información, los drones no necesitan estar diseñados para comunicarse entre sí, lo que simplifica el sistema en general.
- Desventaja:
  - ✓ Los drones no saben qué partes de la finca han cubierto los otros. No pueden aprender a evitar esas áreas.



COBERTURA

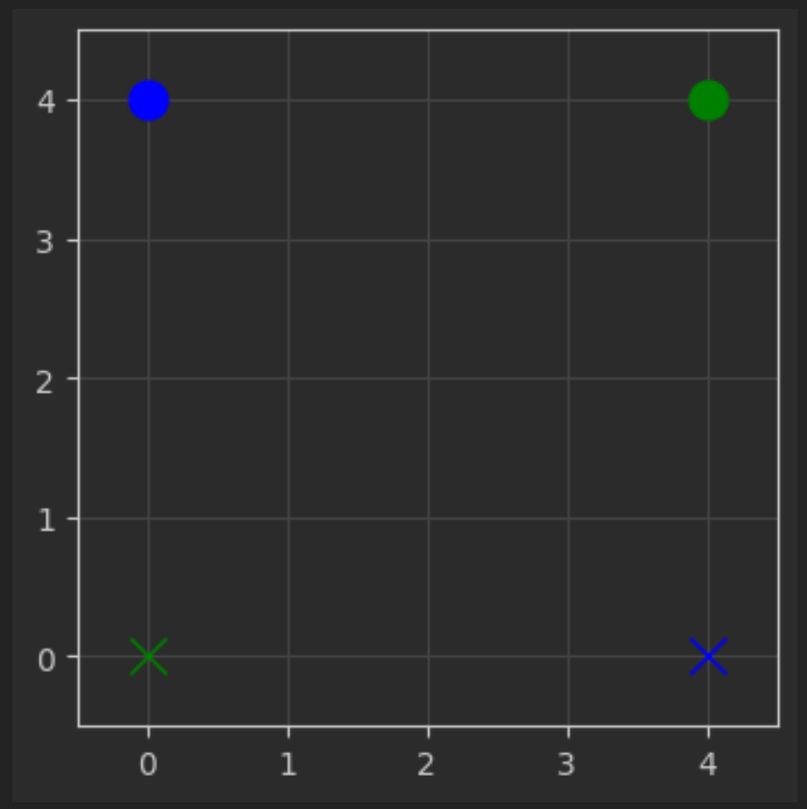
- El dron amarillo piensa que hay un lugar no cubierto a su derecha y otro a su izquierda.
- Si realiza una acción hacia la derecha, debería recibir una recompensa positiva, ya que aún no ha sido cubierto.
- Si ese lugar ya estuviera cubierto por otro dron, entonces sería penalizado.
- Por lo tanto, querrá aprender a evitar esta situación pero el problema es que, como el dron amarillo solo tiene acceso a su propio mapa de cobertura, en realidad no puede aprender a distinguir entre un cuadrado no visitado y uno que fue cubierto por otro dron.





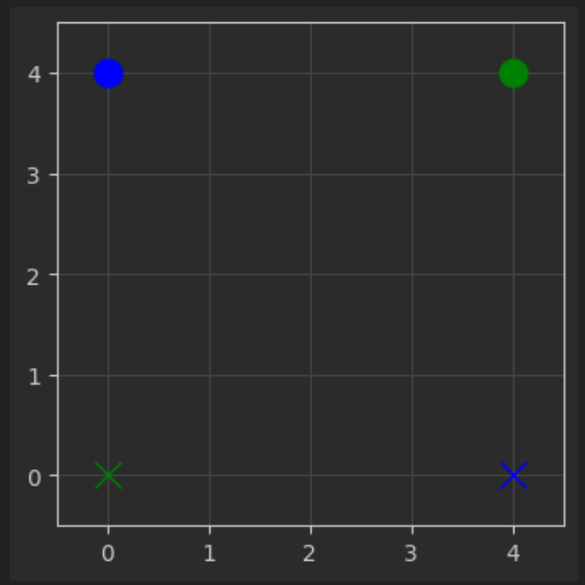
ARQUITECTURA DECENTRALIZADA CON Q-LEARNING

- Dos agentes (azul y verde) empiezan en esquinas superiores opuestas.
- Cada uno debe llegar a su propia meta (esquinas inferiores opuestas).
- Aprenden usando Q-learning independiente. Cada uno aprende su propia política con RL.

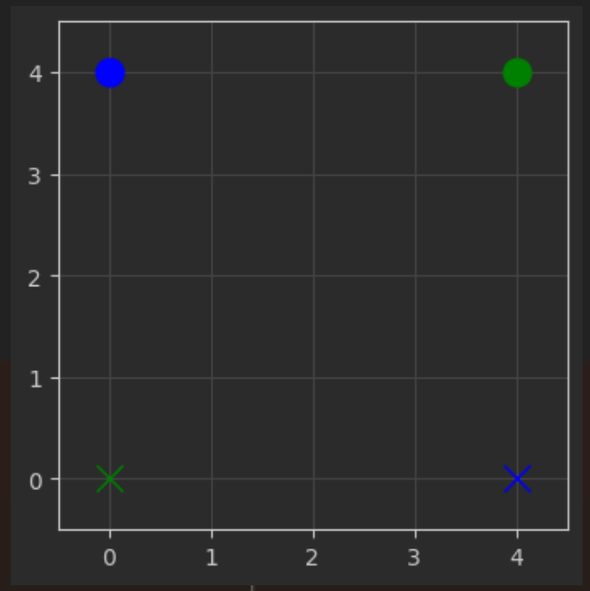


ARQUITECTURA DECENTRALIZADA CON Q-LEARNING

- Interacción: Independiente (sin cooperación ni competencia)
- Recompensas: Individuales (cada agente recibe su propia recompensa)
- Objetivos: Independientes (cada uno tiene su propia meta)
- Observabilidad: Totalmente observable (cada agente "ve" todo el entorno)
- Comunicación: Ninguna
- Aprendizaje: Independiente Q-learning
- Acciones: Sincronizadas (ambos actúan al mismo tiempo)

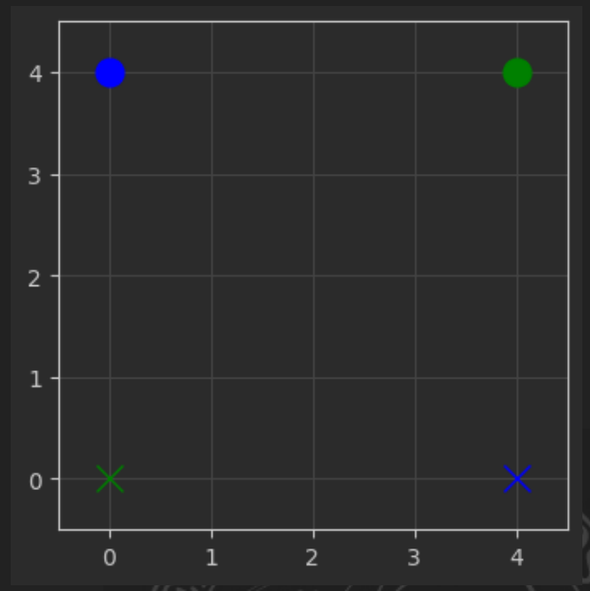


ENTORNO




```python
class GridWorld:
    def __init__(self, size=5):
        self.size = size
        self.reset()

    def reset(self):
        self.agent_pos = [np.array([0, 0]), np.array([0, self.size - 1])]
        self.goals = [np.array([self.size - 1, self.size - 1]), np.array([self.size - 1, 0])]
        return tuple(map(tuple, self.agent_pos))

    def step(self, actions):
        rewards = []
        for i in range(2):
            move = actions[i]
            pos = self.agent_pos[i] + move
            pos = np.clip(pos, 0, self.size - 1)
            self.agent_pos[i] = pos
            rewards.append(1 if np.array_equal(pos, self.goals[i]) else -0.1)
        return tuple(map(tuple, self.agent_pos)), rewards
```

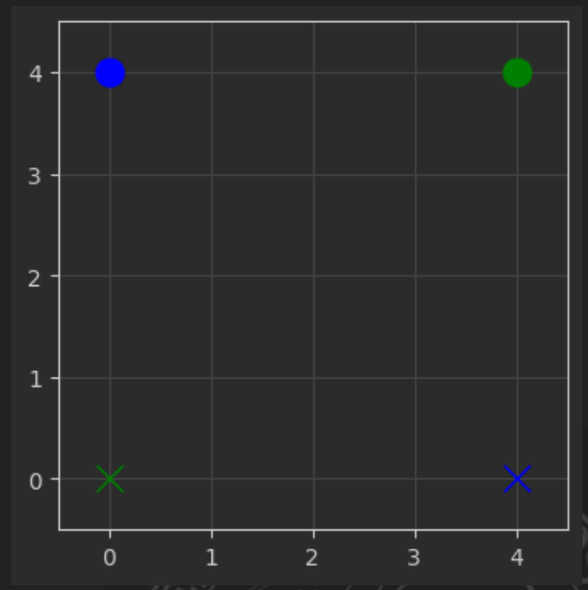
ENTORNO




```python
def state_to_str(state):
    return str(state[0]) + '_' + str(state[1])

Q1, Q2 = {}, {}

def get_q(Q, state):
    return Q.setdefault(state_to_str(state), np.zeros(len(ACTIONS)))

def choose_action(Q, state, epsilon):
    if random.random() < epsilon:
        return random.choice(ACTION_IDX)
    return np.argmax(get_q(Q, state))
```

ENTORNO

```
env = GridWorld()
episodes = 500
alpha = 0.1
gamma = 0.95
epsilon = 1.0
```




```
for ep in range(episodes):
    state = env.reset()
    for _ in range(50):
        a1 = choose_action(Q1, state, epsilon)
        a2 = choose_action(Q2, state, epsilon)
        next_state, rewards = env.step([ACTIONS[a1], ACTIONS[a2]])
        q1 = get_q(Q1, state)
        q2 = get_q(Q2, state)
        q1[a1] += alpha * (rewards[0] + gamma * np.max(get_q(Q1, next_state)) - q1[a1])
        q2[a2] += alpha * (rewards[1] + gamma * np.max(get_q(Q2, next_state)) - q2[a2])
        state = next_state
        if env.is_done():
            break
    epsilon = max(0.05, epsilon * 0.995)
```

CON BIBLIOTECAS

Agentes

RLlib

Entornos

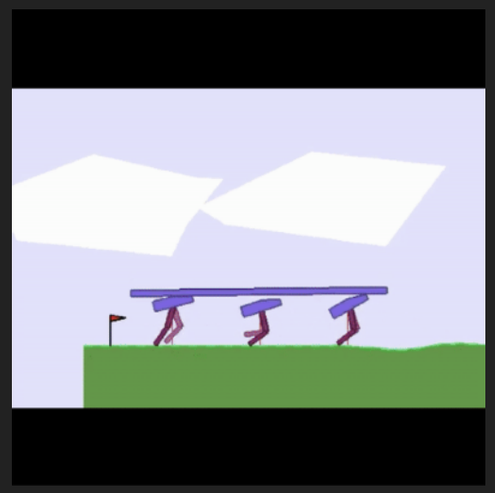
PettingZoo

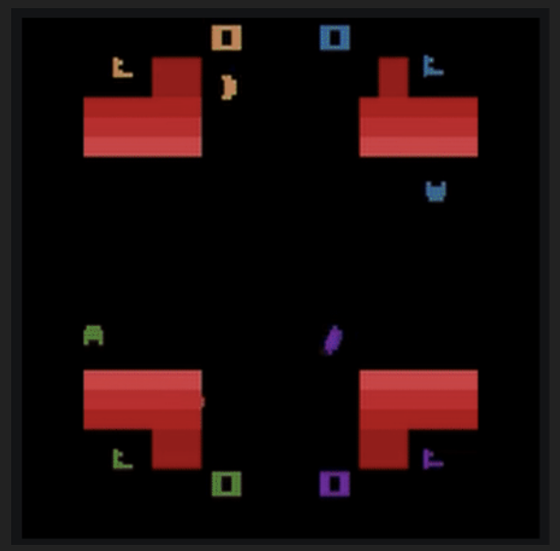
RLIB

- Propósito: Entrenamiento escalable de RL distribuido y producción (cluster).
- Uso principal: Aplicaciones de RL a gran escala.
- Características clave:
  - ✓ Esta basada en Ray (el motor que permite computación distribuida).
  - ✓ Tolera entrenamiento paralelo y multinodo.
  - ✓ Ray incluye Tune (ajuste de hiperparámetros) y Serve (despliegue de modelos).
- Agentes que provee: DQN, A3C (deprecated), PPO, SAC, TD3, DDPG, IMPALA, APEX.
- Puede trabajar con múltiples agentes.

PETTINGZOO

- Propósito: Proporcionar entornos para Multi-Agent Reinforcement Learning (MARL).
- Uso principal: Investigación y entrenamiento en escenarios con múltiples agentes.
- Características clave:
  - ✓ API estándar similar a Gym pero adaptada a multi-agente.
  - ✓ Compatible con Stable-Baselines, RLlib, y otros frameworks.
  - ✓ Soporta entornos paralelos y secuenciales.





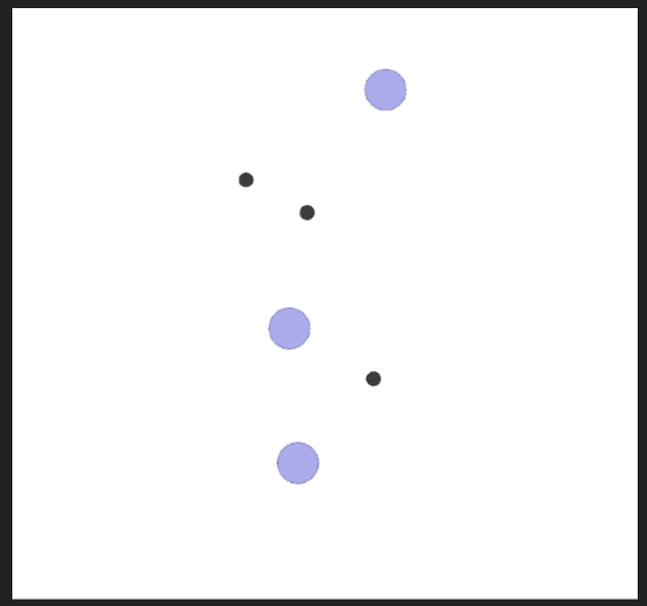
OTROS ENTORNOS EN PETTINGZOO

PettingZoo

- Clásicos (classic)
  - ✓ tic_tac_toe_v3
  - ✓ chess_v5
  - ✓ checkers_v4
  - ✓ connect_four_v3
- Juegos de Atari (atari)
  - ✓ pong_v3, boxing_v5, combat_v5, etc. (con 2 agentes)
- Juegos de tablero (butterfly)
  - ✓ knights_archers_zombies_v10
  - ✓ cooperative_pong_v5

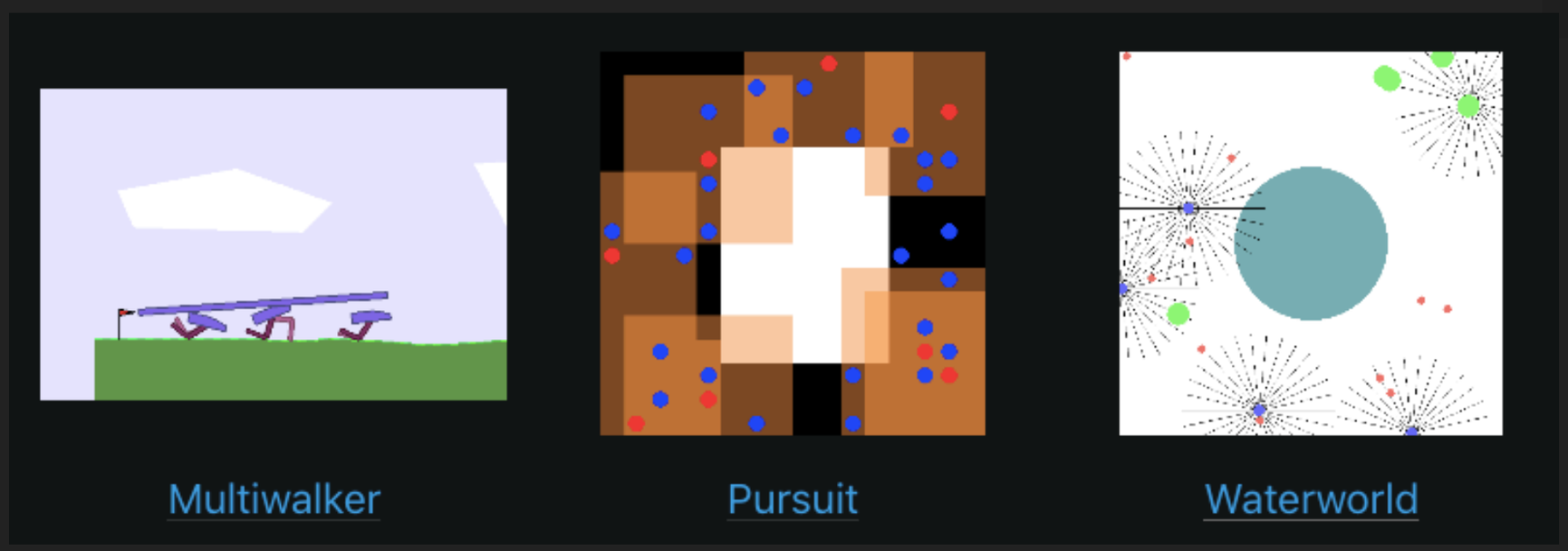
OTROS ENTORNOS EN PETTINGZOO

- MPE (mpe) → entornos con comunicación opcional
  - ✓ simple_v2
  - ✓ simple_spread_v2
  - ✓ simple_reference_v2



OTROS ENTORNOS EN PETTINGZOO

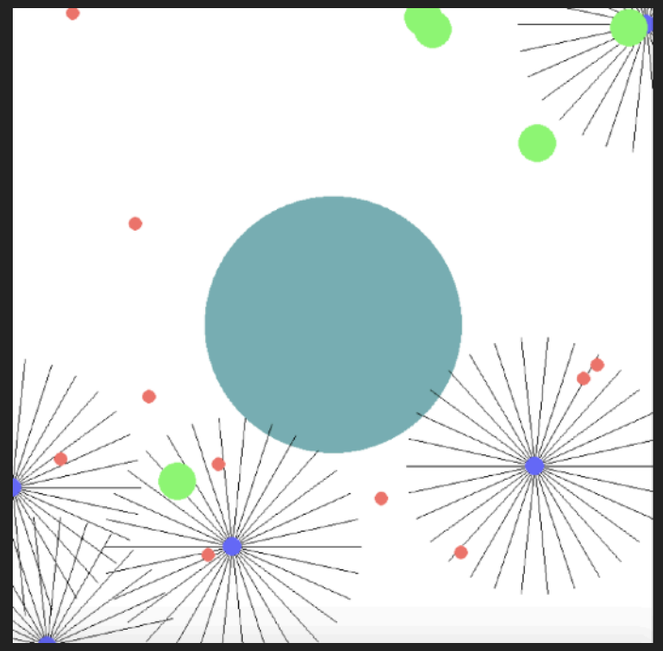
- SISL (sisl) → entornos continuos multiagente cooperativos
  - ✓ waterworld_v4
  - ✓ multiwalker_v9
  - ✓ pursuit_v4



WATERWORLD

PettingZoo

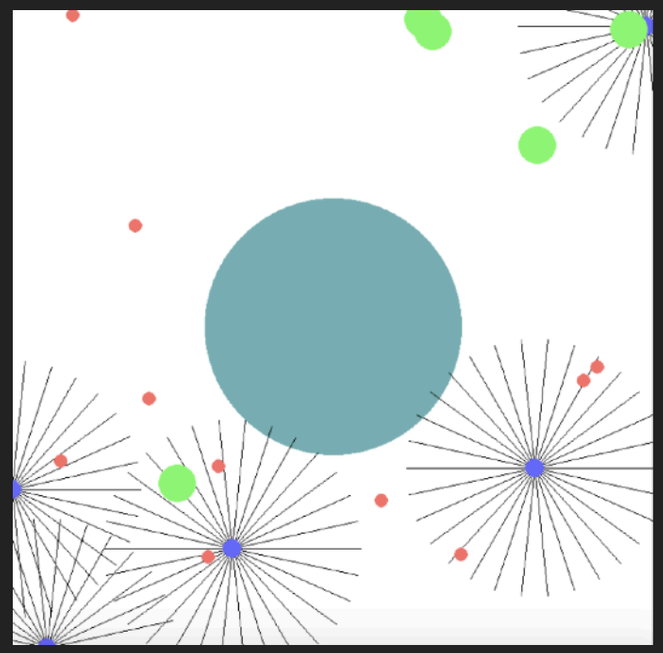
- Agentes. denominados pursuers (perseguidores).
  - ✓ Navegan para recolectar comida (objetivos verdes).
  - ✓ Evitan el veneno (objetivos rojos).
  - ✓ Operan en un espacio limitado, con posiciones en coordenadas x e y dentro del rango [0,1].



WATERWORLD – CARACTERÍSTICAS TÉCNICAS

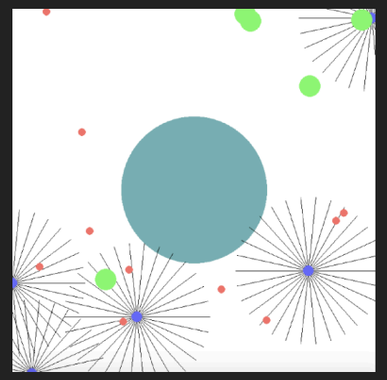
PettingZoo

- Agentes: 5 pursuers (por defecto)
- Acciones: Espacio continuo de 2 dimensiones (empuje horizontal y vertical)
- Observaciones: Vectores de 242 elementos que incluyen información de sensores sobre distancias y velocidades de obstáculos, barreras, comida, veneno y otros agentes, además de indicadores de colisión con comida o veneno.
- Sensores: Cada agente cuenta con 30 sensores distribuidos uniformemente que detectan objetos cercanos.
- Duración del episodio: 500 pasos por defecto



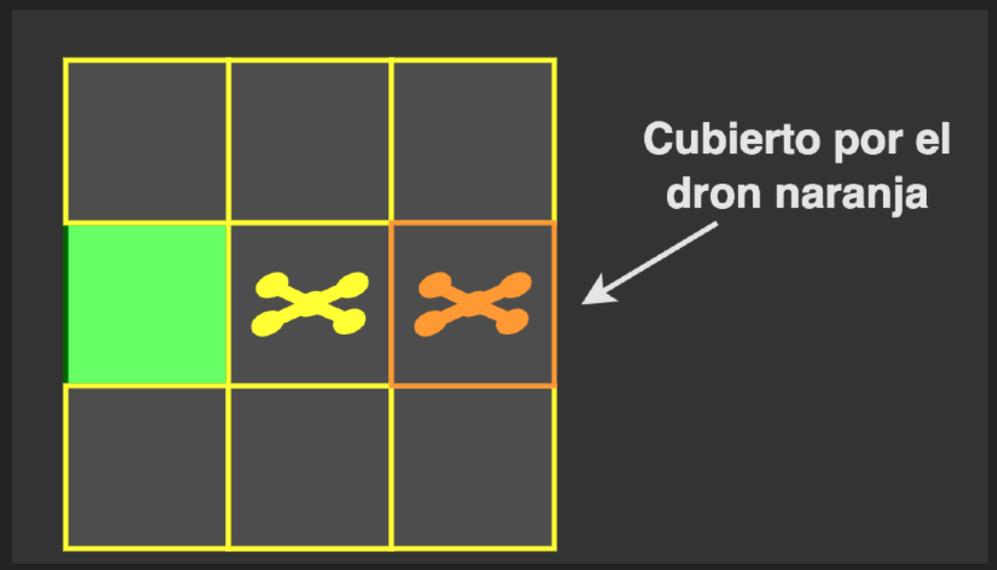
WATERWORLD – CONFIGURACIÓN DE PPO

```
config = (
    PPOConfig()
    .environment(env=env_name, clip_rewards=True)
    .framework("torch")
    .rollouts(num_rollout_workers=0)
    .training(
        model={"fcnet_hiddens": [64, 64]}
    )
    .multi_agent(
        policies={
            agent: (None, dummy_env_rllib.observation_space[agent], dummy_env_rllib.action_space[agent], {})
            for agent in agents
        },
        policy_mapping_fn=(lambda agent_id, episode, worker, **kwargs: agent_id),
    )
    .resources(num_gpus=0)
    .debugging(log_level="WARN")
)
```



COBERTURA

- Solución (posible) –> cada uno de los drones debería compartir su mapa de cobertura con cada dron.
  - ✓ Esto requiere hardware o software adicional que permita este intercambio de información.
  - ✓ En este caso debe considerarse el balance entre el rendimiento y la complejidad general del proyecto.



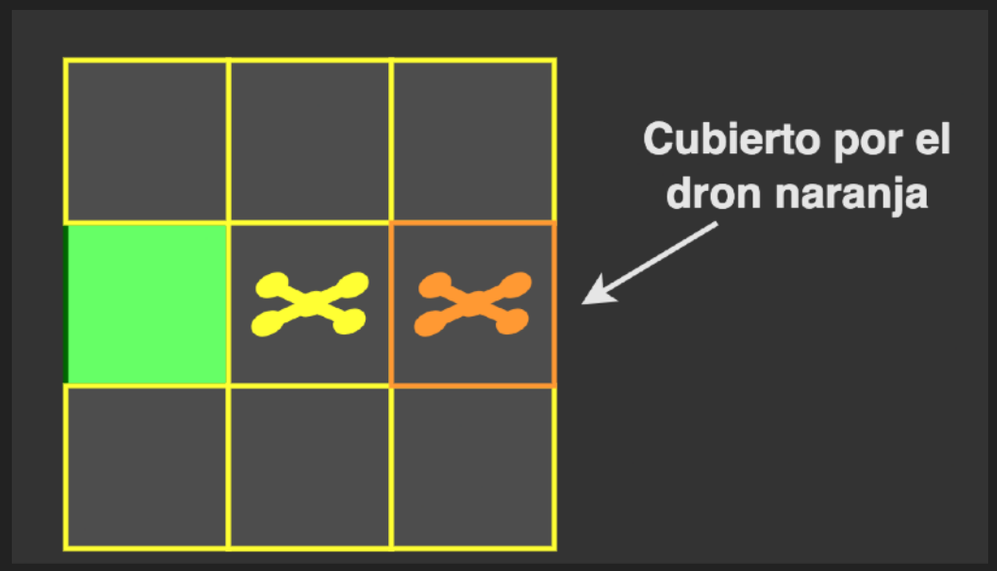
WATERWORLD COOPERATIVO

- En este caso la recompensa es compartida por los agentes.

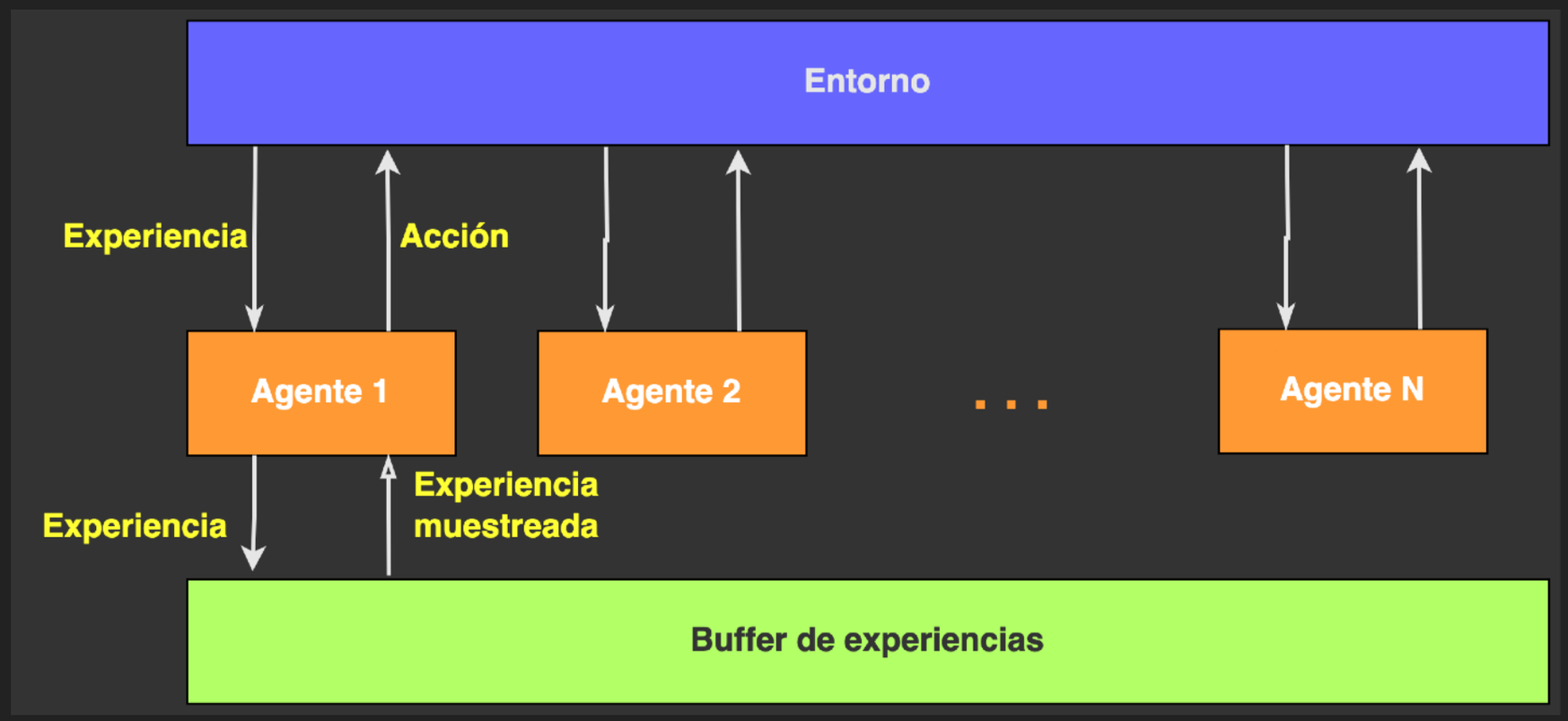
```
ppo_config = (
    PPOConfig()
    .environment(env=ENV_NAME, clip_rewards=True)
    .framework("torch")
    .rollouts(num_rollout_workers=1)
    .training(model={"fcnet_hiddens": [128, 128]})
    .multi_agent(
        policies={
            "shared_policy": (
                None,
                dummy_rllib_env.observation_space[agents[0]],
                dummy_rllib_env.action_space[agents[0]],
                {}
            )
        },
        policy_mapping_fn=(lambda agent_id, episode, worker, **kwargs: "shared_policy")
    )
    .resources(num_gpus=0)
    .debugging(log_level="WARN")
)
```

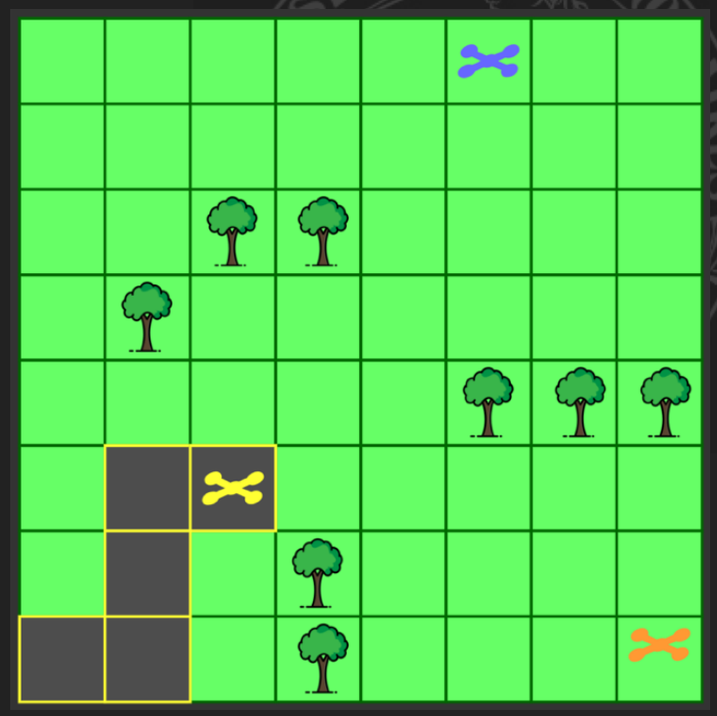
COBERTURA

- Otra solución (posible) –> el intento de ocupar el mismo espacio por ambos drones hace que puedan colisionar –> entorno no estacionario.
  - ✓ Esto requiere hardware o software adicional que permita este intercambio de información.
  - ✓ En este caso debe considerarse el balance entre el rendimiento y la complejidad general del proyecto.



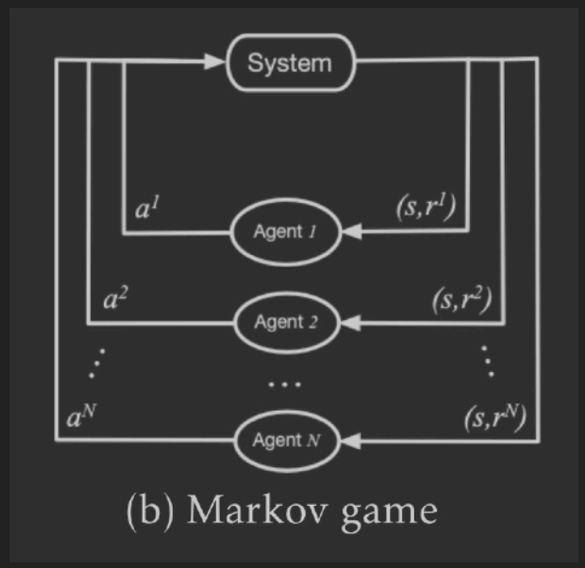
ESCENARIO 2 – ARQUITECTURA CENTRALIZADA





JUEGOS ESTOCÁSTICOS Y JUEGOS EN FORMA EXTENSIVA

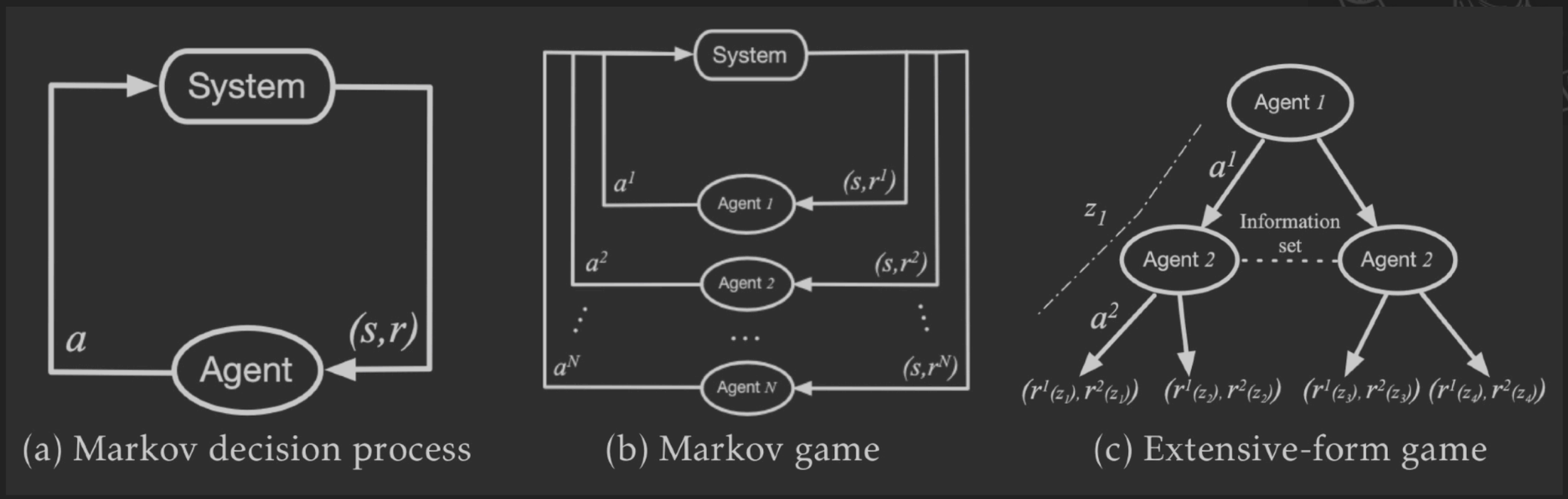
- EL MDP que representa la interacción de múltiples agentes se conoce como juego de Markov, (o juego estocástico) donde:
  - ✓ Cada agente $i \in N$ en el tiempo t ejecuta una acción a_{it} para el estado s_t del sistema.
  - ✓ El sistema luego transita al estado s_{t+1}, y recompensa a cada agente i con la recompensa $R^i_{at}(s_t, s_{t+1})$.
  - ✓ El objetivo del agente i es optimizar su propia recompensa a largo plazo, encontrando la política $\pi_i : S \rightarrow E(Ai)$ como un mapeo desde el espacio de estados a una distribución sobre el espacio de acciones, tal que $a_{it} \sim \pi_i(\cdot|st)$.
  - ✓ Entonces, la función de valor $V_i : S \rightarrow R$ del agente i se convierte en una función de la política conjunta $\pi : S \rightarrow E(A)$ definida como $\pi(a|s) = \Pi_{i \in N} \pi^i(a^i|s)$.



(b) Markov game

JUEGOS ESTOCÁSTICOS Y JUEGOS EN FORMA EXTENSIVA

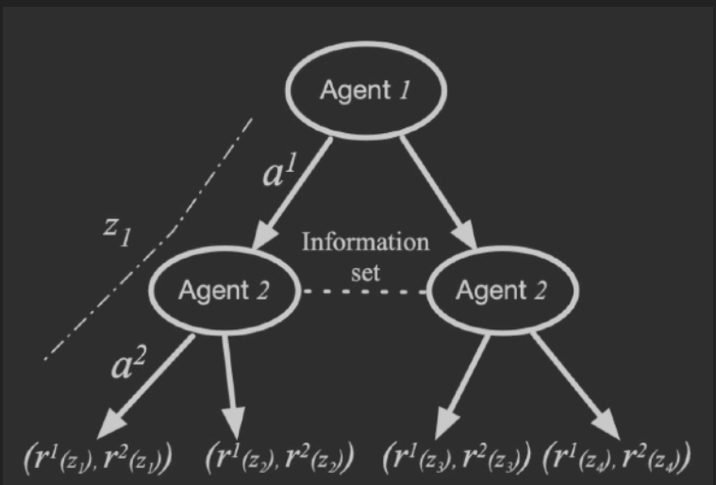
- La información imperfecta en múltiples agentes se diagrama como en (c) (Zhang et al., 2021):
  - ✓ (a) Diagrama agente/entorno clásico.
  - ✓ (b) Versión multiagente para juegos de Markov.
  - ✓ (c) Árbol de juego en forma extensiva. Un árbol de juego modela la información imperfecta.



(a) Markov decision process (b) Markov game (c) Extensive-form game

JUEGOS ESTOCÁSTICOS Y JUEGOS EN FORMA EXTENSIVA

- Las elecciones del agente se muestran como enlaces sólidos, y la información privada oculta del otro agente se muestra con una linea discontinua.
- Los agentes toman decisiones al elegir acciones a^i, y reciben su recompensa individual $r^i(z)$ al final del juego, donde z es la puntuación de un nodo terminal para esa rama.
- El conjunto de información se indica con una línea punteada, para señalar el comportamiento estocástico; el entorno (u otro agente) elige un resultado desconocido entre las acciones punteadas.
- La notación en forma extensiva está diseñada para expresar juegos de información imperfecta, donde todos los resultados posibles (desconocidos) se representan en el conjunto de información.
- Para calcular la función de valor, el agente debe considerar todas las posibles elecciones diferentes en los conjuntos de información; las elecciones desconocidas de los oponentes crean un gran espacio de estados.



(c) Extensive-form game

ESTRATEGIAS COMPETITIVAS, COOPERATIVAS Y MIXTAS

- MARL se dividen en 3 tipos de problemas, es decir, con comportamiento:
  - ✓ Competitivo. La recompensa de los agentes suma cero. Una victoria para un agente es una pérdida para otro.
  - ✓ Cooperativo. Los agentes colaboran para optimizar una recompensa común a largo plazo. Una victoria para un agente es una victoria para todos los agentes.
  - ✓ Mixto. Involucra tanto agentes cooperativos como competitivos, con las llamadas recompensas de suma general (general-sum rewards); algunas acciones pueden llevar a situaciones de ganar/ganar, otras a ganar/perder.

COMPORTAMIENTO COMPETITIVO

- Teoría de Juegos (von Newmann et al., 1944)
- Puntos de equilibrio en juegos de N personas (Nash, 1950)

John von Newmann

John Forbes Nash

COMPORTAMIENTO COMPETITIVO

- Teoría de Juegos
- Puntos de equilibrio en juegos de N personas

TEORÍA DE JUEGOS

- La teoría de juegos (rama de matemática/economía):
  - ✓ Estudia cómo toman decisiones los agentes racionales cuando sus resultados dependen de lo que otros también decidan.
  - ✓ Analiza estrategias óptimas en situaciones donde hay conflicto, cooperación o competencia entre varios actores racionales.

TEORÍA DE JUEGOS

- La teoría de juegos establece que:
  - ✓ Los jugadores eligen estrategias para maximizar su beneficio o minimizar su pérdida.
  - ✓ Cada jugador tiene en cuenta las posibles decisiones de los demás.

TEORÍA DE JUEGOS

- ¿Qué aporta la Teoría de Juegos?
  - ✓ Modela desde dilemas sociales hasta estrategias económicas, guerras, negociaciones, biología evolutiva, etc.
  - ✓ Predice comportamientos racionales o por qué a veces cooperamos aunque no sea lo más egoístamente rentable.

TEORÍA DE JUEGOS – TIPOS DE JUEGOS

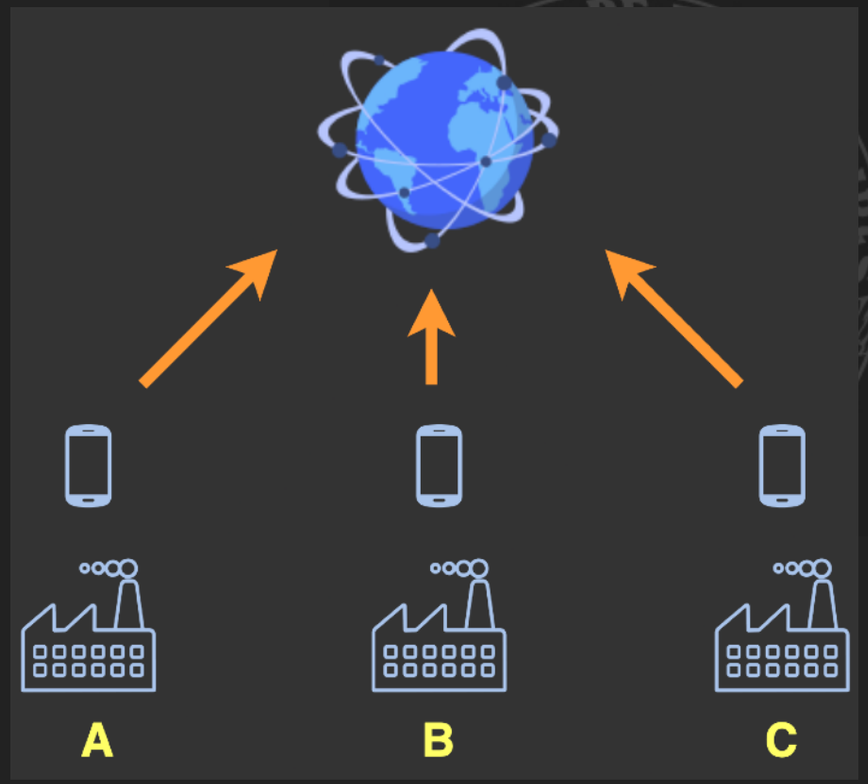
- Un juego es de información perfecta si:
  - ✓ En cada momento del juego, todos los jugadores conocen completamente el estado del juego y todas las acciones pasadas que se han tomado.
- Un juego de información imperfecta es aquel en el que:
  - ✓ Al menos un jugador no conoce completamente lo que ha pasado antes (acciones, decisiones, estados ocultos).

COMPORTAMIENTO COMPETITIVO

- Teoría de Juegos
- Puntos de equilibrio en juegos de N personas

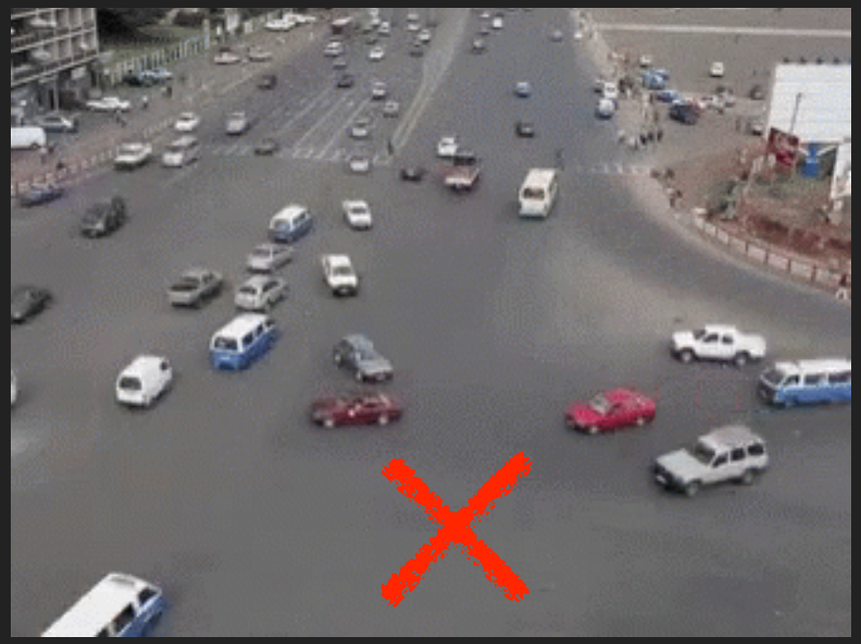
EQUILIBRIO DE NASH

- Ejemplo: 3 empresas que venden un mismo producto compiten por un mercado.
  - ✓ Cada empresa elige su estrategia (precio, publicidad, calidad, etc.) intentando maximizar su beneficio.
  - ✓ No hay acuerdo entre ellas (es decir, no cooperan).
  - ✓ Cada una fija un precio.
  - ✓ Si una empresa baja demasiado el precio, pierde margen de ganancia.
  - ✓ Si lo sube, pierde clientes frente a las otras.
- En un Equilibrio de Nash:
  - ✓ Cada empresa fija un precio considerando lo que hacen las otras.
  - ✓ A cada empresa no le conviene cambiar el precio si las demás mantienen el suyo.



EQUILIBRIO DE NASH

- Equilibrio de Nash: Se define como aquella situación en la que ningún agente (player) tiene nada que ganar cambiando su propia estrategia. Para dos agentes competitivos, el equilibrio de Nash es la estrategia minimax.
- Define las condiciones para una solución estable (y en cierto sentido óptima) entre múltiples agentes racionales no cooperativos.



EQUILIBRIO DE NASH EN MARL

- En MARL, el objetivo es encontrar aquella multipolítica (política combinada) que para cada agente maximiza su recompensa futura acumulada.
- El conjunto de estrategias competitivas (casi) de nula explotabilidad se conoce como equilibrio de Nash.

COMPORTAMIENTO COMPETITIVO

- El equilibrio de Nash caracteriza un punto de equilibrio π^*, desde el cual ninguno de los agentes tiene incentivo para desviarse. Esto es, para cualquier agente $i \in N$, la política $\pi^{i,*}$ es la mejor respuesta a $\pi^{-i,*}$, donde $-i$ son todos los agentes excepto i.

COMPORTAMIENTO COMPETITIVO

- Fin Parte 1 -

REFERENCIAS BIBLIOGRÁFICAS Y WEB (I)

- https://pettingzoo.farama.org/environments/sisl/
- Plaat, A. (2022). Deep reinforcement learning (Vol. 10, pp. 978-981). Singapore: Springer.
- Zhang, K., Yang, Z., & Başar, T. (2021). Multi-agent reinforcement learning: A selective overview of theories and algorithms. Handbook of reinforcement learning and control, 321-384.
- Von Neumann, J. (1928). Zur theorie der gesellschaftsspiele. Mathematische annalen, 100(1), 295-320.
- Von Neumann, J., & Morgenstern, O. (1944). Theory of Games and Economic Behavior. Princeton University Press.
- Nash Jr, J. F. (1950). Equilibrium points in n-person games. Proceedings of the national academy of sciences, 36(1), 48-49.